

Privora

Technical Handover Document

Prepared for: Bassam — Founder, Privora
Document Type: Full Technical Handover
Date: April 2026
Classification: Confidential

Table of Contents

1. Full System Walkthrough
2. What Has Been Built
3. Deployment & Infrastructure
4. Database & Data Handling
5. Security Architecture
6. Integration Capability
7. DevSecOps & CI/CD
8. WAF & Infrastructure Security
9. Mobile Readiness
10. Scaling & Future Architecture
11. Team & Future Development
12. Documentation & Ownership

1. Full System Walkthrough

Architecture Overview

Privora is a three-tier web application:

```
Browser (React SPA)
  ↓ HTTPS / REST API
Express.js API Server (Node.js) – hosted on Railway
  ↓ Mongoose ODM
MongoDB Atlas (Cloud Database)
```

Layer	Technology	Hosting
Frontend	React 18 + TypeScript + Vite + Tailwind CSS	GitHub Pages (demo) / Vercel (production)
Backend	Node.js + Express.js	Railway
Database	MongoDB Atlas	MongoDB Cloud

End-to-End Data Flow — Consent Example

1. Org admin creates a consent request via the dashboard form.
2. Frontend sends POST /api/consents with title, dataTypes, purpose, legalBasis, retentionPeriod.
3. Backend validates the JWT token, confirms the user is org_admin, saves to the database.
4. User logs into their portal, fetches GET /api/consents/requests.
5. User approves — POST /api/consents/:id/respond with status: "approved".
6. Backend creates a ConsentResponse record and writes an immutable AuditLog entry.
7. Org admin dashboard refreshes and shows updated approval rate.

Module Interaction Map

Consent Management

- └─ triggers → Data Sharing Gateway (consent required before share)
- └─ triggers → Audit Log (every action logged)
- └─ triggers → Privacy Notice (notice version recorded)

Data Sharing Gateway

- └─ requires → Consent approval
- └─ writes → Audit Log (every transfer logged)
- └─ checks → Purpose Registry (purpose validation)

DSAR Module

- └─ references → Consent records
- └─ writes → Audit Log
- └─ enforces → 30-day SLA deadline

Breach Management

- └─ writes → Audit Log
- └─ enforces → 72-hour authority notification (PDPL Art. 19)

Compliance Dashboard

- └─ reads all modules for aggregated KPIs and score

2. What Has Been Built

Module Summary

Module	Status	Key Capability
Authentication	Complete	OTP login, JWT tokens, RBAC
Consent Management	Complete	Create, approve, reject, withdraw
Data Sharing Gateway	Complete	Inter-org data exchange with consent gate
DSAR	Complete	5 request types, 30-day SLA
Breach Management	Complete	Incident log, 72-hour deadline tracking
Data Retention	Complete	Policy per data category, expiry tracking
Privacy Notices	Complete	Versioned notices, activation control
Purpose Registry	Complete	PDPL Art. 5 validation, purpose mapping
Audit Logs	Complete	Immutable append-only log
Analytics	Complete	Approval rate, response breakdown, KPIs
Compliance Dashboard	Complete	PDPL Score, overdue alerts
User Portal	Complete	WOW dashboard, activity timeline
Tests	156 passing	Integration tests for all backend modules

Key Business Rules

- Consent cannot be used for data sharing unless status is `approved`.
- DSAR SLA is calculated from `createdAt` and flagged overdue after 30 days.

- Breach authority deadline is 72 hours from `discoveredAt`.
- Consent withdrawal sets status to `withdrawn` and blocks future data sharing under that consent.
- All audit log entries are append-only — no update or delete routes exist.
- Organization data is scoped by `organizationId` on every query.

Current Limitations

Email Delivery: OTP is generated but email delivery requires an SMTP provider (SendGrid / AWS SES) to be configured. In demo mode, OTP is returned in the API response.

AES-256 Encryption: MongoDB Atlas provides encryption at rest on paid tiers (M10+). There is no explicit application-level field encryption in the current code. See Section 5 for full detail.

File Transfer: Data sharing is metadata-based — no actual file upload or binary transfer system exists yet.

Background Jobs: Expiry checks and SLA calculations happen at query time, not via a scheduler. This is fine at low scale.

3. Deployment & Infrastructure

Current Setup

Component	Service	Notes
Frontend	GitHub Pages	Demo only — use Vercel for production
Backend API	Railway	Auto-deploys on GitHub push
Database	MongoDB Atlas	Connection string stored in Railway env vars

Required Environment Variables

```
--- Backend (Railway) ---
MONGODB_URI      = mongodb+srv://...
JWT_SECRET       = <strong random string, 64+ chars>
JWT_REFRESH_SECRET = <strong random string, 64+ chars>
OTP_SECRET       = <strong random string>
CLIENT_URL      = https://your-frontend-domain.com
NODE_ENV         = production
PORT            = 3000

--- Frontend (build time) ---
VITE_API_URL     = https://your-backend.railway.app/api
VITE_DEMO_MODE   = false
```

Step-by-Step Production Deployment

- MongoDB Atlas:** Create a cluster, whitelist Railway's IP address, copy the connection string.
- Railway:** Connect your GitHub repo to Railway. Add all environment variables in the Railway dashboard. Railway auto-deploys on every push to main.
- Frontend:** Set VITE_API_URL to your Railway backend URL. Run `npm run build`. Deploy the `dist` folder to Vercel or Netlify.
- CORS:** Update the allowed origins in the backend to include your production frontend domain.

5. **Test:** Register an org, create a consent request, respond as a user, verify audit log entry.

4. Database & Data Handling

Collections (Tables)

Collection	Key Fields
users	_id, name, email, role, organizationId, otp, otpExpiry, refreshToken
organizations	_id, name, logoUrl, contactEmail, isActive
consent_requests	organizationId, title, dataTypes[], purpose, legalBasis, retentionPeriod, expiresAt, isActive
consent_responses	consentRequestId, userId, status, reason, respondedAt
data_sharing_requests	organizationId, requestingOrgId, targetUserId, dataTypes[], status
data_sharing_transactions	requestId, organizationId, userId, dataTypes[], action
dsar_requests	userId, organizationId, requestType, status, slaDeadline, isOverdue
breach_incidents	organizationId, severity, status, discoveredAt, timeline[], notifiedAuthority, authorityDeadline, isAuthorityOverdue
retention_policies	organizationId, dataCategory, retentionPeriodDays, action, legalBasis
privacy_notices	organizationId, title, content, version, isActive
processing_purposes	organizationId, name, legalBasis, dataTypes[], pdplArticle, isActive
audit_logs	userId, organizationId, action, resource, resourceId, metadata, ipAddress, createdAt

Data Isolation

Every database query includes an `organizationId` filter. Org admins can only see their organization's data. Users can only see records

targeting their userId. This is enforced at the controller (code) level on every request.

Backup Strategy

Recommended (production): MongoDB Atlas M10+ with continuous backup (point-in-time recovery).

Manual backup command:

```
mongodump --uri="mongodb+srv://..." --out=./backup-YYYY-MM-DD
```

Restore command:

```
mongorestore --uri="mongodb+srv://..." ./backup-YYYY-MM-DD
```

5. Security Architecture

Authentication Flow

1. User submits their email address.
2. Backend generates a 6-digit OTP, stores a hashed version + expiry on the user record, sends OTP to email.
3. User submits the OTP code.
4. Backend verifies hash and expiry, issues a JWT access token (15 min) and refresh token (7 days).
5. Frontend stores the access token in memory and refresh token in localStorage.
6. Every API call includes: `Authorization: Bearer <accessToken>`
7. On 401 response, frontend automatically uses the refresh token to get a new access token.

RBAC Permissions

Action	user	org_admin
View own consent requests	✓	✓
Approve / reject / withdraw consent	✓	✗
Create consent requests	✗	✓
View org analytics	✗	✓
Process DSAR requests	✗	✓
Log breach incidents	✗	✓
Manage retention policies	✗	✓
View all org audit logs	✗	✓

AES-256 Encryption — Honest Assessment

In Transit: All traffic uses HTTPS (TLS 1.2/1.3). Enforced automatically by Railway and GitHub Pages. ✓

At Rest (Atlas level): MongoDB Atlas M10+ clusters encrypt all data at rest with AES-256 by default. The free tier (M0) does not guarantee this. Use M10+ for production. ✓ (with correct tier)

At Application Level: There is no explicit field-level encryption in the current codebase. Personal data fields are stored in plaintext in MongoDB, protected by access controls and Atlas-level encryption. If enterprise clients require field-level encryption, implement MongoDB CSFLE (Client-Side Field Level Encryption).

API Security Measures in Place

- Rate limiting on authentication endpoints
- CORS restricted to the frontend domain
- Input validation on all POST/PUT routes
- JWT verification middleware on all protected routes
- Organization scoping on every data query

6. Integration Capability

API Structure

All endpoints are RESTful JSON under the `/api` prefix. Standard response format:

```
Success: { "success": true, "data": { ... } }
Error:   { "success": false, "message": "Description" }
```

Key Endpoints Summary

Method	Endpoint	Description
POST	<code>/api/auth/send-otp</code>	Request OTP
POST	<code>/api/auth/verify-otp</code>	Verify OTP, receive tokens
POST	<code>/api/auth/refresh</code>	Refresh access token
GET	<code>/api/consents/requests</code>	User's pending consent requests
POST	<code>/api/consents</code>	Create consent request (org_admin)
POST	<code>/api/consents/:id/respond</code>	User responds (approve/reject)
POST	<code>/api/data-sharing/requests</code>	Create data sharing request
POST	<code>/api/data-sharing/requests/:id/respond</code>	User approves/rejects share
GET/ POST	<code>/api/dsar/requests</code>	Submit / list DSAR requests
GET/ POST	<code>/api/breaches</code>	Breach management
GET	<code>/api/analytics</code>	Org compliance analytics
GET	<code>/api/audit-logs</code>	Audit log viewer

How External Systems Connect

1. Register an organization and admin account via the auth flow.

2. Obtain a JWT token via OTP verification.
3. Pass the token in every request: `Authorization: Bearer <token>`
4. Call any API endpoint in JSON.

Future Integration Possibilities

Integration	What's Needed
SDAIA Breach Notification	Add outbound POST call when breach is reported to authority
National ID Verification (Absher/Nafath)	Add identity check step during user registration
HR Systems	Add webhook support to trigger events in third-party systems
Mobile Apps	No backend changes needed — API is already mobile-ready

7. DevSecOps & CI/CD

Current State

There is no automated CI/CD pipeline in place. Deployment is currently manual:

- **Backend:** Push to GitHub → Railway auto-deploys automatically.
- **Frontend:** Run `npm run build` locally → run `npx gh-pages -d dist`.

Recommended GitHub Actions Pipeline

On push to main branch:

1. Install dependencies (`npm ci`)
2. Run backend tests (`npm test`) – must all pass
3. TypeScript check (`tsc --noEmit`)
4. Build frontend (`npm run build`)
5. Deploy frontend to Vercel via Vercel CLI
(Railway handles backend deployment automatically)

Security in the Pipeline

Check	Tool	Status
Dependency vulnerabilities	npm audit	Add to pipeline
Secret scanning	GitHub built-in	Enable on repo
Static code analysis (SAST)	Semgrep / Snyk	Add to pipeline
Integration tests	Jest (156 tests)	Complete
Dynamic testing (DAST)	OWASP ZAP	Not yet

8. WAF & Infrastructure Security

Recommended Setup — Cloudflare

Place Cloudflare in front of the entire system. Point your custom domain to Railway through Cloudflare DNS (enable proxy / orange cloud). This provides:

- DDoS protection
- Web Application Firewall (blocks SQLi, XSS, path traversal)
- Rate limiting at the network edge
- SSL termination
- Bot detection and challenge pages
- Real-time traffic analytics

Application-Level Security Actions

Risk	Mitigation	Status
NoSQL Injection	Add express-mongo-sanitize middleware	Must add
XSS	React JSX escapes output by default. No dangerouslySetInnerHTML used.	Protected
API Rate Abuse	Apply express-rate-limit globally (currently only on auth)	Extend
CORS Misconfiguration	Restrict to exact production domain only	In place
Insecure HTTP Headers	Add helmet middleware	Verify
HTTP Parameter Pollution	Add hpp middleware	Add

9. Mobile Readiness

The backend is fully ready for mobile app integration today. It is a stateless REST API returning JSON — any iOS (Swift), Android (Kotlin), or React Native app can use the same endpoints with no backend changes required.

Recommended Additions Before Mobile Launch

Feature	What's Needed
Push Notifications	Add Firebase Cloud Messaging (FCM). Store device token on user document. Trigger on new consent requests, DSAR updates, breach alerts.
Biometric Auth	Handled client-side — device biometric unlocks the stored JWT. No backend changes.
API Versioning	Introduce <code>/api/v1/</code> prefix now to avoid breaking changes later.
Token Storage	Mobile apps must use iOS Keychain / Android Keystore — not <code>localStorage</code> .

10. Scaling & Future Architecture

Capacity Estimate

Scale	What's Needed
0–100 organizations	Current setup — no changes required
100–500 organizations	Redis caching, DB indexes, Railway scale-up
500+ / Enterprise	Horizontal scaling, background jobs, API gateway
Government level	AWS/Azure KSA region (data residency), isolated DB per client

Key Additions at Scale

- **Redis caching** — cache analytics and consent lists with 60-second TTL.
- **Background job processor** — Bull (Redis-backed) for async tasks: OTP emails, SLA checks, retention expiry runs.
- **Database indexing** — add compound indexes on (organizationId, createdAt) for all main collections.
- **API Gateway** — Kong or AWS API Gateway for auth offloading, analytics, and rate limiting at scale.
- **Data residency** — migrate to AWS Riyadh or Azure UAE North for PDPL data localization compliance.

11. Team & Future Development

Recommended Team by Phase

Phase	Clients	Team Needed
Phase 1	0–10	1 Senior Full-Stack Developer + Founder
Phase 2	10–50	Backend Eng + Frontend Eng + Part-time DevOps + Part-time Security
Phase 3	50+	Add QA Engineer + Data Engineer + Solutions Engineer

Immediate Priorities Before First Client

1. Configure email delivery (SendGrid or AWS SES) for OTP
2. Upgrade MongoDB Atlas to M10 tier
3. Add express-mongo-sanitize middleware
4. Set up a staging environment on Railway
5. Enable GitHub secret scanning
6. Set up uptime monitoring (BetterUptime or UptimeRobot)
7. Write an incident runbook (what to do if backend goes down)

12. Documentation & Ownership

Repository Structure

/backend	
/src	
/models	– Mongoose schemas (all data structures)
/controllers	– Business logic per module
/routes	– Express route definitions
/middleware	– Auth, RBAC, error handling
/tests	
/integration	– 156 integration tests
server.js	– Entry point
/frontend	
/src	
/pages	
/user	– User portal pages
/org	– Organization admin pages
/layouts	– OrgLayout, UserLayout
/store	– Zustand auth state
/lib	– API client, demo mode interceptors
/components	– Shared UI components

Documentation Gaps to Close

Item	Priority	Tool
API documentation (interactive)	High	swagger-jsdoc + swagger-ui-express → /api/docs
Setup / onboarding guide	High	README.md in repository
Admin runbook	Medium	Notion or Confluence
Data dictionary	Medium	Document all field meanings

Credentials & Access

Security Rule: Never share credentials (MongoDB URI, JWT secrets, Railway tokens) over email or chat. Use a password manager such as 1Password or Bitwarden for secure sharing.

- **GitHub:** Repository at OxBassam/Bassam — branch: claude/build-privora-mvp-OLBP4

- **Railway:** Managed through your Railway account — env vars set in the dashboard
- **MongoDB Atlas:** Managed through your Atlas account — connection string in Railway env vars only

13. Action Items Before Production

1. Email Delivery — Configure SendGrid or AWS SES so OTP emails are actually delivered to users.

2. MongoDB Atlas Tier — Upgrade to M10+ to guarantee AES-256 encryption at rest.

3. NoSQL Injection — Add `express-mongo-sanitize` middleware to the backend.

4. API Documentation — Add Swagger/OpenAPI docs at `/api/docs` before developer onboarding.

5. CI/CD Pipeline — Set up GitHub Actions to automate testing and deployment.

6. Staging Environment — Deploy a separate staging instance on Railway for pre-production testing.

7. Monitoring & Alerts — Configure uptime monitor and Railway alerts so downtime is caught immediately.

8. Data Residency — Confirm whether enterprise clients require data hosted in a Saudi Arabia cloud region. If yes, plan AWS Riyadh / Azure UAE North migration before signing contracts.